

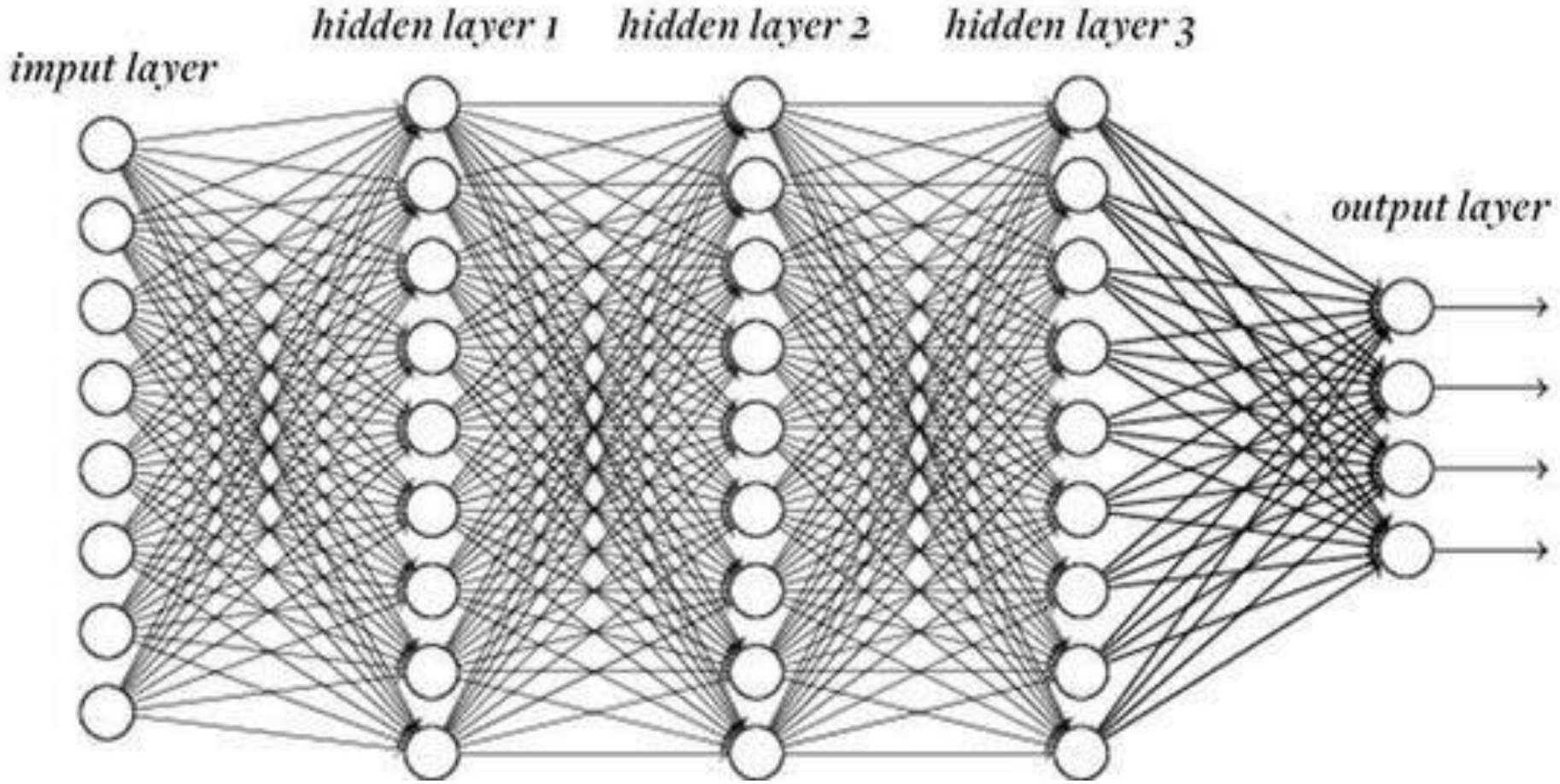
# Topic 6 深度学习
## Part 1：深度神经网络

魏准
浙江大学，信息与电子工程学院
2026

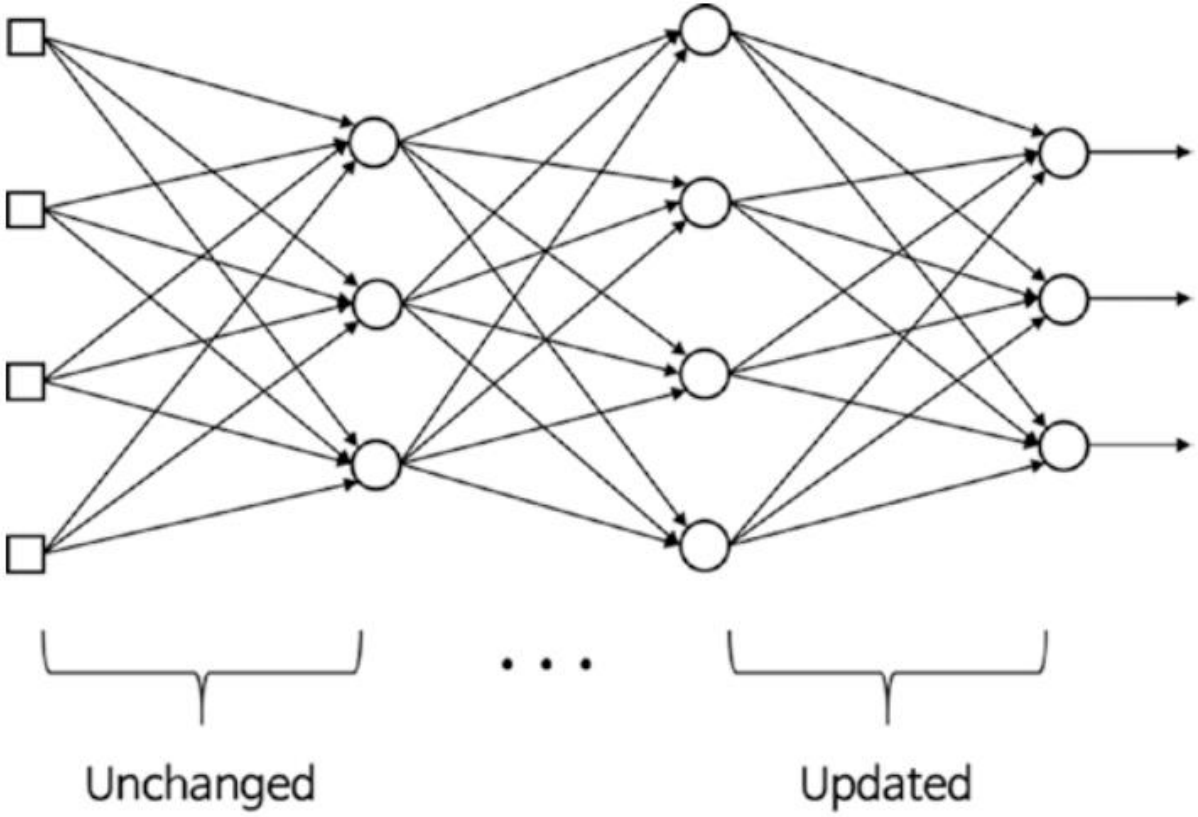
# 总体内容

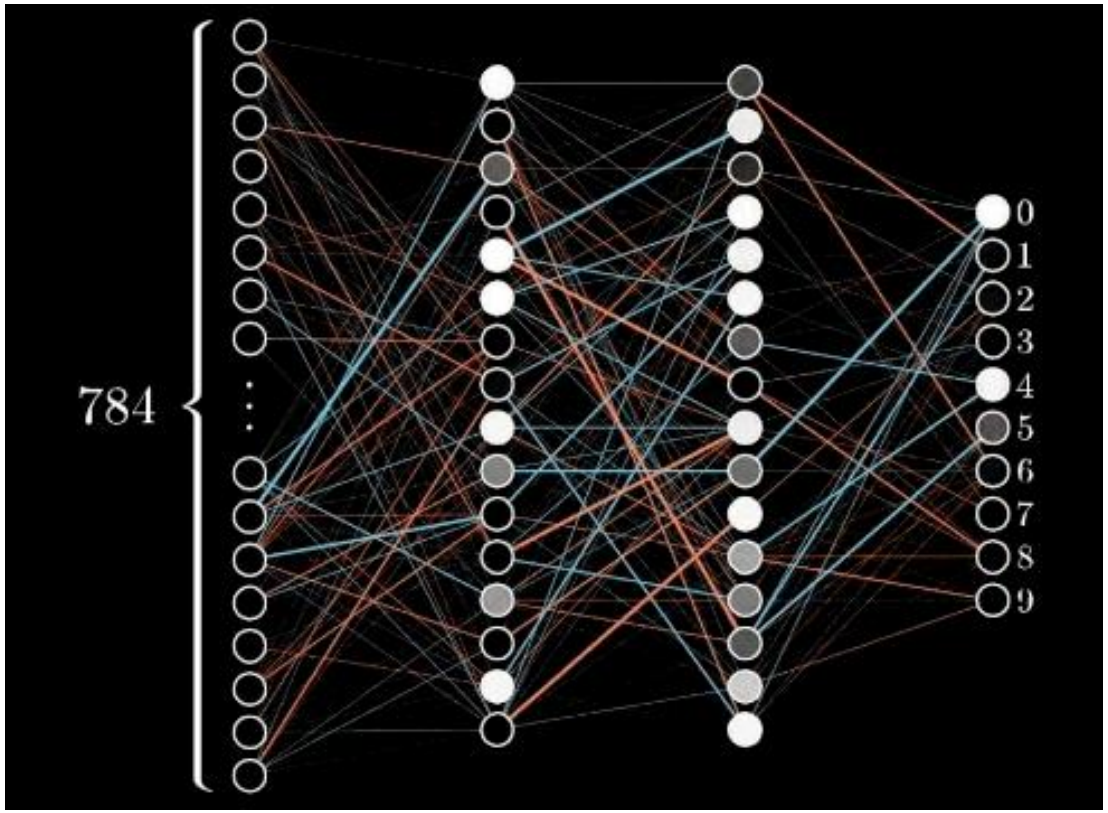
主要知识点

- 梯度消失(Vanishing Gradient)；
- 实例：训练一个深度学习网络；
- 实验6-1；
- 过拟合，Dropout；
- 实验6-2；


imput layer
hidden layer 1
hidden layer 2
hidden layer 3
output layer

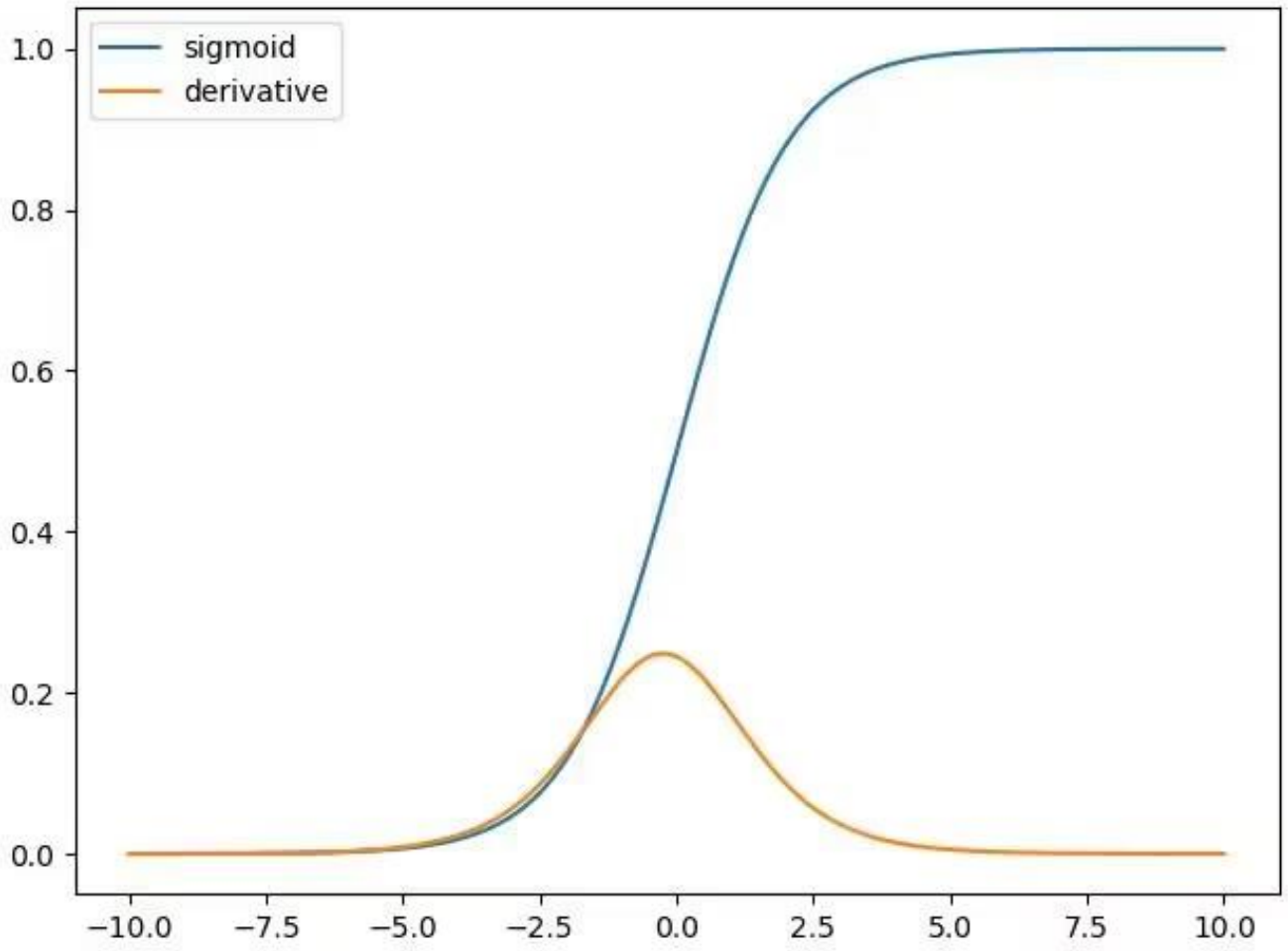
# 1.1：梯度消失(Vanishing Gradient)



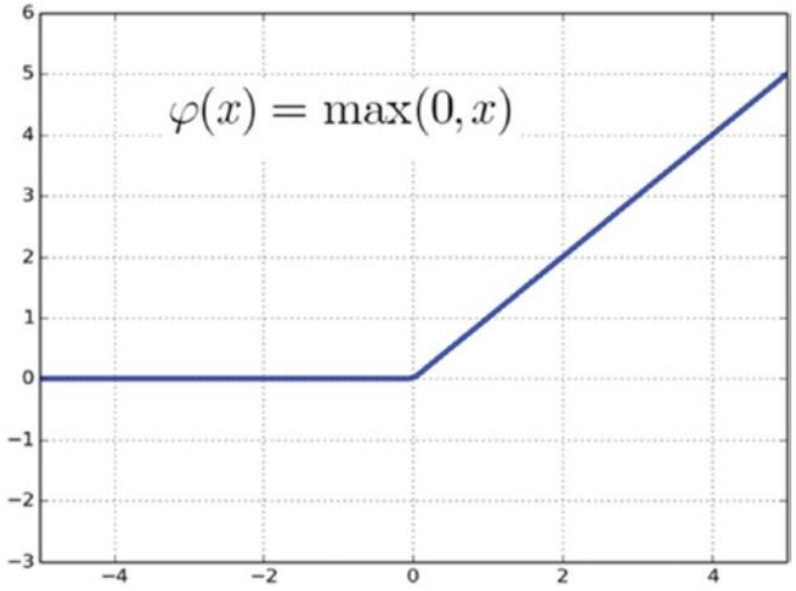
# 1.1：梯度消失(Vanishing Gradient)

解决办法：ReLu激活函数

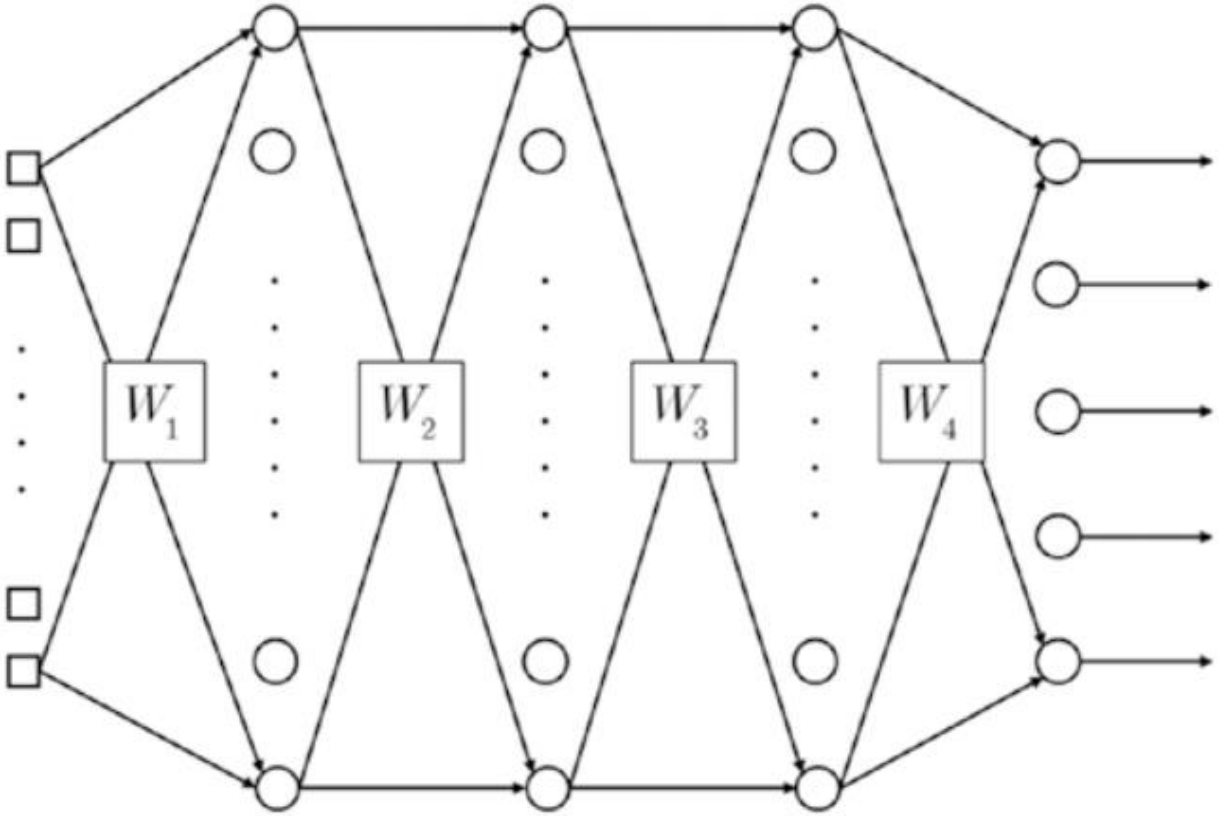



$$\varphi(x) = \begin{cases} x, & x > 0 \\ 0, & x \le 0 \end{cases} \qquad \varphi'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \le 0 \end{cases}$$

$$= \max(0,\ x)$$

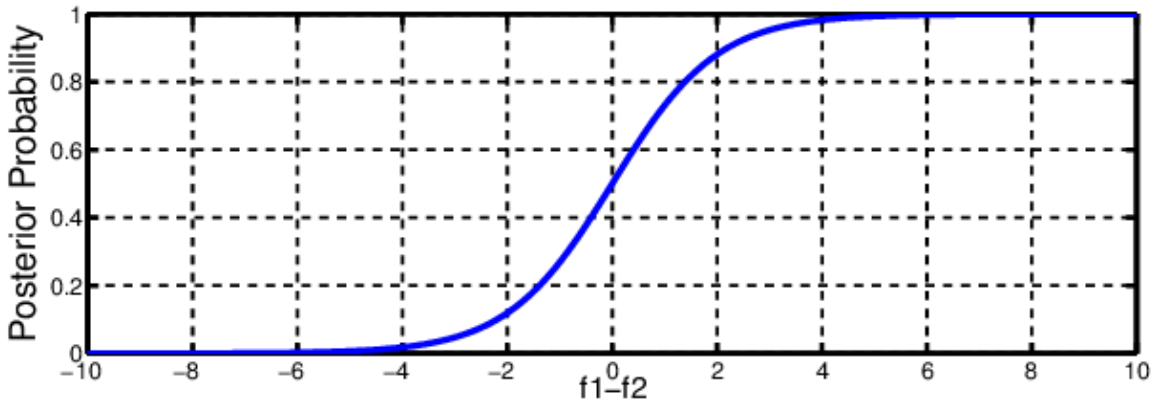


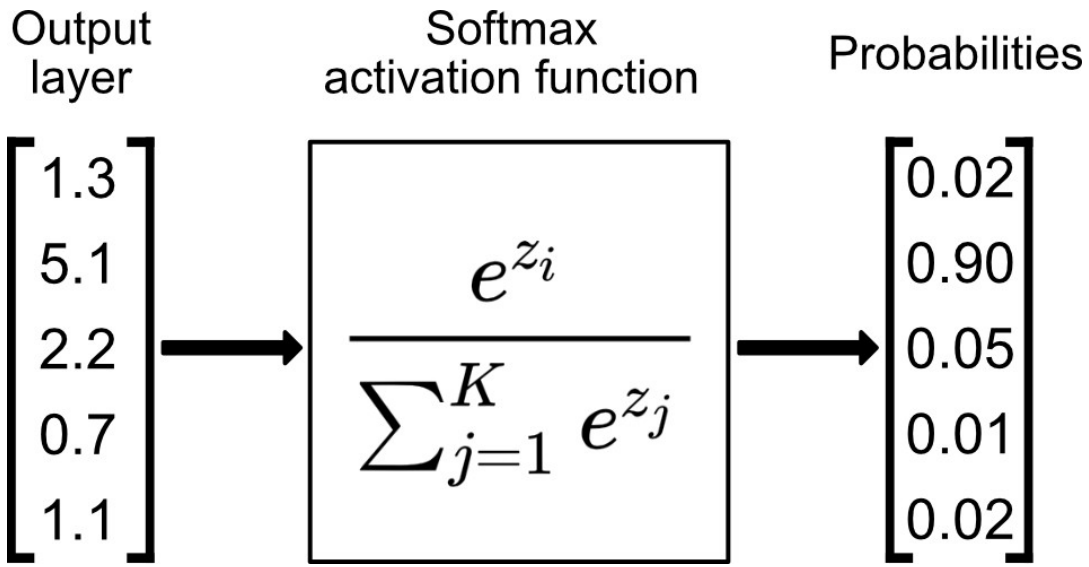
# 1.2：实例：训练一个深度学习网络



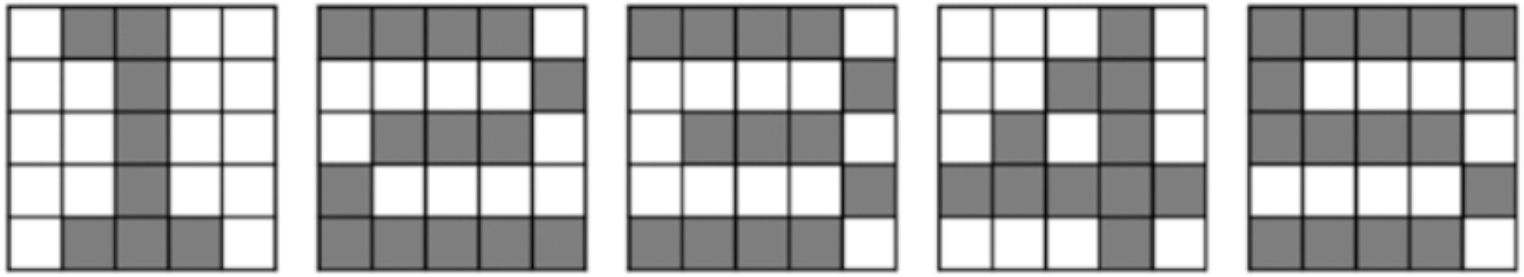

W1： 20*25
W2： 20*20
W3： 20*20
W4： 5*20

激活函数：除最后一层采用softmax外，其他层采用ReLU



训练数据:



```
X(:, :, 1) = [ 0 1 1 0 0;
               0 0 1 0 0;
               0 0 1 0 0;
               0 0 1 0 0;
               0 1 1 1 0
             ];
```

```
X(:, :, 2) = [ 1 1 1 1 0;
               0 0 0 0 1;
               0 1 1 1 0;
               1 0 0 0 0;
               1 1 1 1 1
             ];
```

```
X(:, :, 3) = [ 1 1 1 1 0;
               0 0 0 0 1;
               0 1 1 1 0;
               0 0 0 0 1;
               1 1 1 1 0
             ];
```

```
X(:, :, 4) = [ 0 0 0 1 0;
               0 0 1 1 0;
               0 1 0 1 0;
               1 1 1 1 1;
               0 0 0 1 0
             ];
```

```
X(:, :, 5) = [ 1 1 1 1 1;
               1 0 0 0 0;
               1 1 1 1 0;
               0 0 0 0 1;
               1 1 1 1 0
             ];
```

```
D = [ 1 0 0 0 0;
      0 1 0 0 0;
      0 0 1 0 0;
      0 0 0 1 0;
      0 0 0 0 1
    ];
```

实验6-1：

通过SGD训练方法、ReLu激活函数（ Softmax 最后一层）及BP规则，对上述深度神经网络进行训练，并输出训练后的结果。（为简单起见softmax 函数的导数直接取1）；

## Step 1:定义ReLu、Softmax函数

```
import numpy as np

def ReLU(x):
    return XXXXX

def Softmax(x):
    x  = np.subtract(x, np.max(x))          # prevent overflow
    ex = np.exp(x)
    return ex / np.sum(ex)
```

# Step 2:定义DeepReLU训练函数

```
def DeepReLU(W1, W2, W3, W4, X, D):
    alpha = XXXXX
    N = 5
    for k in range(N):
        x  = np.reshape(X[:, :, k], (25, 1))

        v1 = np.matmul(W1, x)
        y1 = ReLU(v1)
        v2 = XXXXX
        y2 = XXXXX
        v3 = XXXXX
        y3 = XXXXX
        v = XXXXX
        y = XXXXX

        d     = D[k, :].T
        e     = d - y
        delta = e  # softmax for simplicity

        e3     = XXXXX
        delta3 = XXXXX
        e2     = XXXXX
        XXXXX
        dW4 = XXXXX
        W4  = XXXXX
```



工程学院
hejiang University

# Step 3:验证

```
X = np.zeros((5, 5, 5))

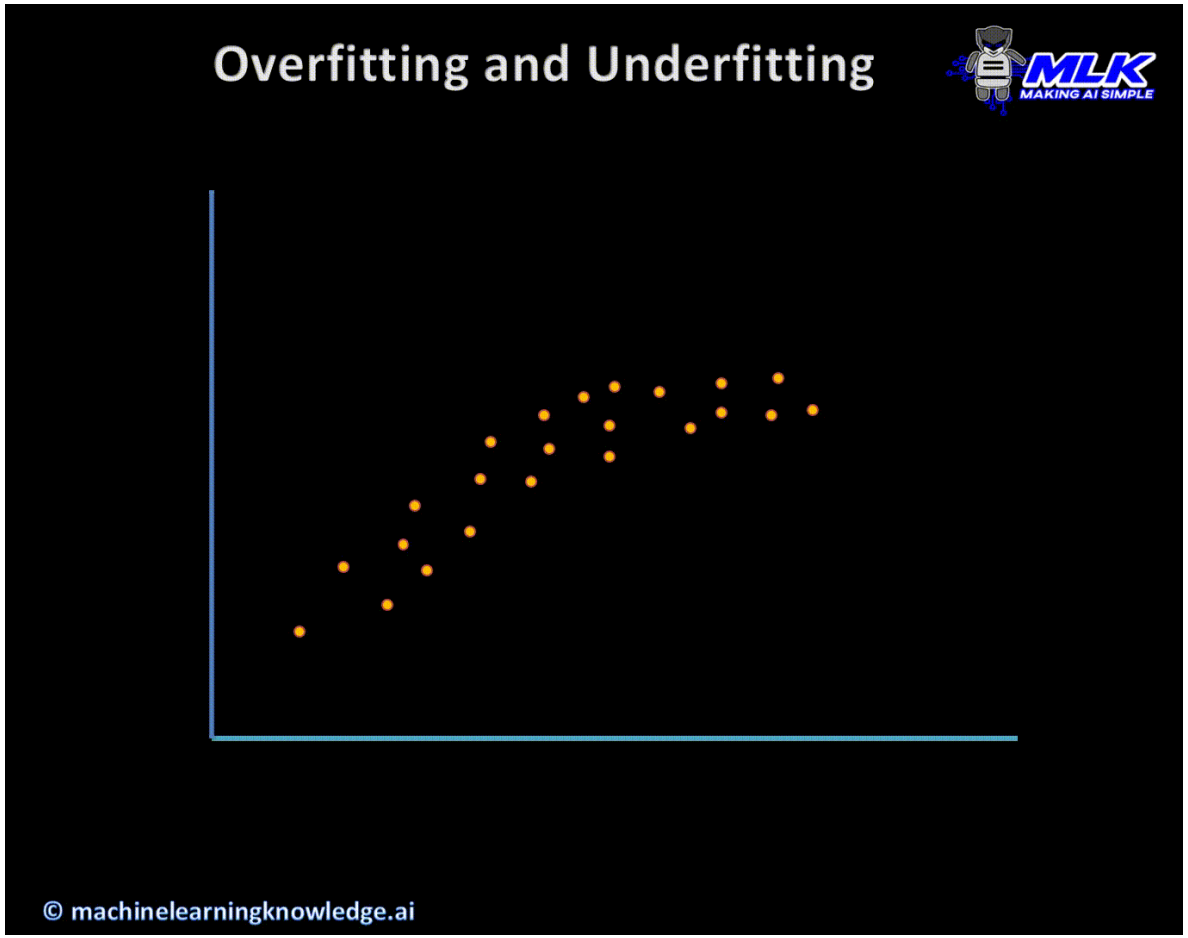
X[:, :, 0] = [[0,1,1,0,0],
              [0,0,1,0,0],
              [0,0,1,0,0],
              [0,0,1,0,0],
              [0,1,1,1,0]]

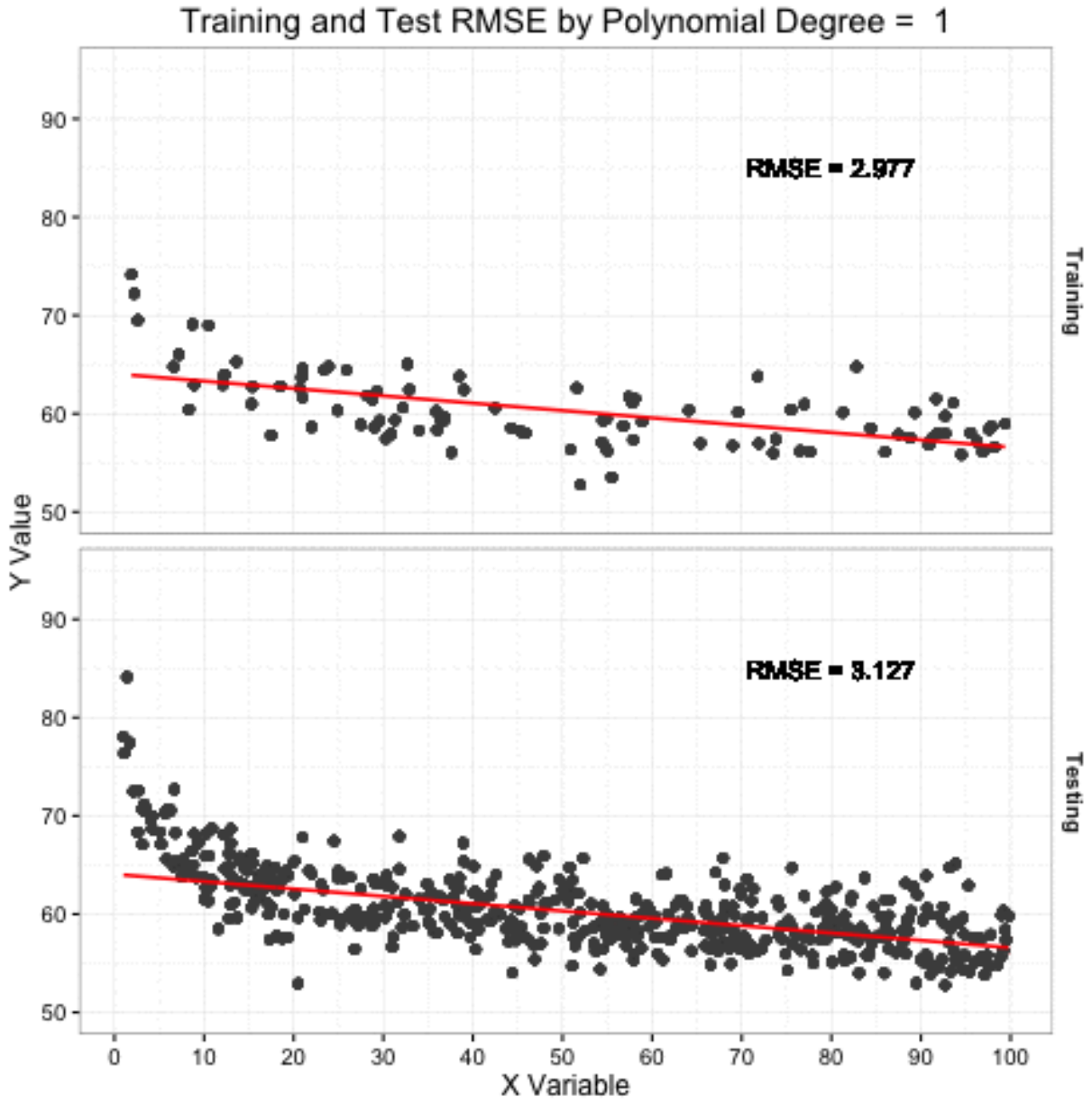
XXXXXXXXXX

D = np.array([[[1,0,0,0,0]],
              [[0,1,0,0,0]],
              [[0,0,1,0,0]],
              [[0,0,0,1,0]],
              [[0,0,0,0,1]]])

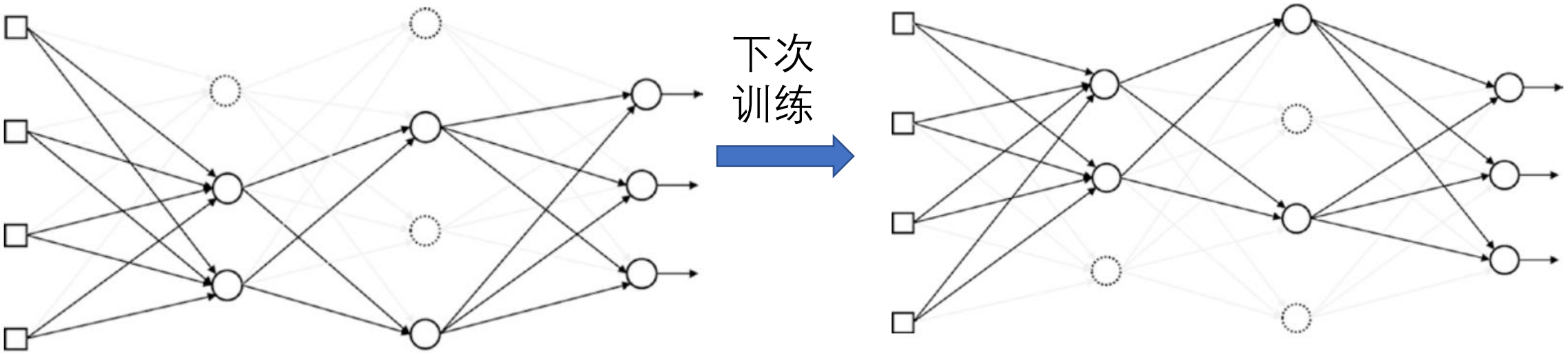
W1 = 2*np.random.random((20, 25)) - 1
XXXXXXXXXX
```

# 1.3：过拟合(Overfitting)



# 1.3：过拟合(Overfitting)




解决办法：Dropout（Dropout_rate：p），剩下的元素乘以1/(1-p)，仅在训练的时候用！

## Step 1:定义Dropout函数

```python
import numpy as np

def Dropout(y, ratio):
    ym = np.zeros_like(y)
    num = round(y.size*(1-ratio))
    idx = np.random.choice(y.size, num, replace=False)
    ym[idx] = 1.0 / (1.0 - ratio)
    return ym
```

$$y_1 = \begin{bmatrix} 0.5356 \\ 0.9537 \\ 0.5442 \\ 0.0821 \\ 0.3663 \\ 0.8509 \end{bmatrix} \quad ym = \begin{bmatrix} 2 \\ 2 \\ 0 \\ 0 \\ 0 \\ 2 \end{bmatrix} \quad y_1 * ym = \begin{bmatrix} 1.0712 \\ 1.9075 \\ 0 \\ 0 \\ 0 \\ 1.7017 \end{bmatrix}$$

# 实验6-2：

- 加入dropout技巧，采用Sigmoid激活函数（Softmax 最后一层）及BP规则，对上述深度神经网络进行训练，并输出训练后的结果。
- 比较加与不加dropout时候MSE vs Epoch变化曲线。

## Step 2:定义DeepDropout训练函数(输出层不加dropout)

```
        v1 = np.matmul(W1, x)
        y1 = ReLU(v1)
        y1 = y1 * Dropout(y1, 0.1)
```

## Step 3:验证

# 附录